

NPU 기반 AI Air Drum / Piano Pad

카메라 기반 손가락 추적과 엣지 NPU를 활용한 저지연 가상 악기 인터페이스 구현

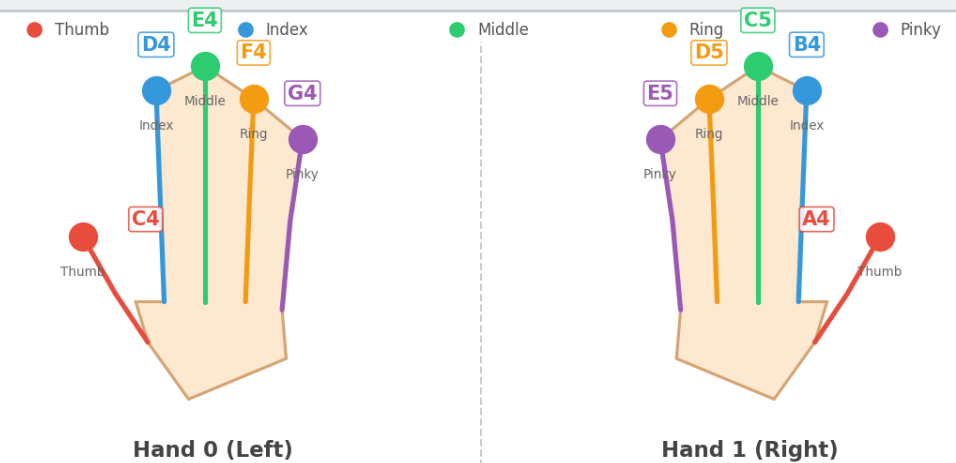
발표 관점

- 코드 사용법보다 시스템 아키텍처와 설계 판단을 중심으로 설명
- CPU와 NPU에 어떤 연산을 배치했는지, 왜 그렇게 나눴는지 제시
- 현재 구현된 실험 루프와 남은 위험을 중간 산출물 관점에서 정리

Piano — Default C Major (Dynamic)

10-slot piano: C4-E5 pentatonic, distance-adaptive range

```
$ python3 main.py --piano --camera 0
```



발표에서 답해야 하는 질문

1. 무엇을 만들었는가?

손가락 landmark를 이용해 실제 타격 동작을 인식하고, 드럼/피아노 사운드를 재생하는 가상 악기 인터페이스.

2. NPU는 어디에 쓰는가?

현재 안정 경로에서는 hand landmark CNN을 NPU `.dxnn`으로 실행하고, palm detection은 CPU TFLite로 유지한다.

3. 왜 그렇게 분할했는가?

Palm detector는 INT8 양자화 후 score head가 깨졌다. 반면 hand landmark는 NPU 가속 효과가 확인됐다.

4. 어느 정도 검증했는가?

라이브 데모 경로, dataset replay benchmark, landmark error 비교, 단위 테스트, 보정 실험을 갖췄다.

SYSTEM REQUIREMENTS

시스템 요구사항과 제약

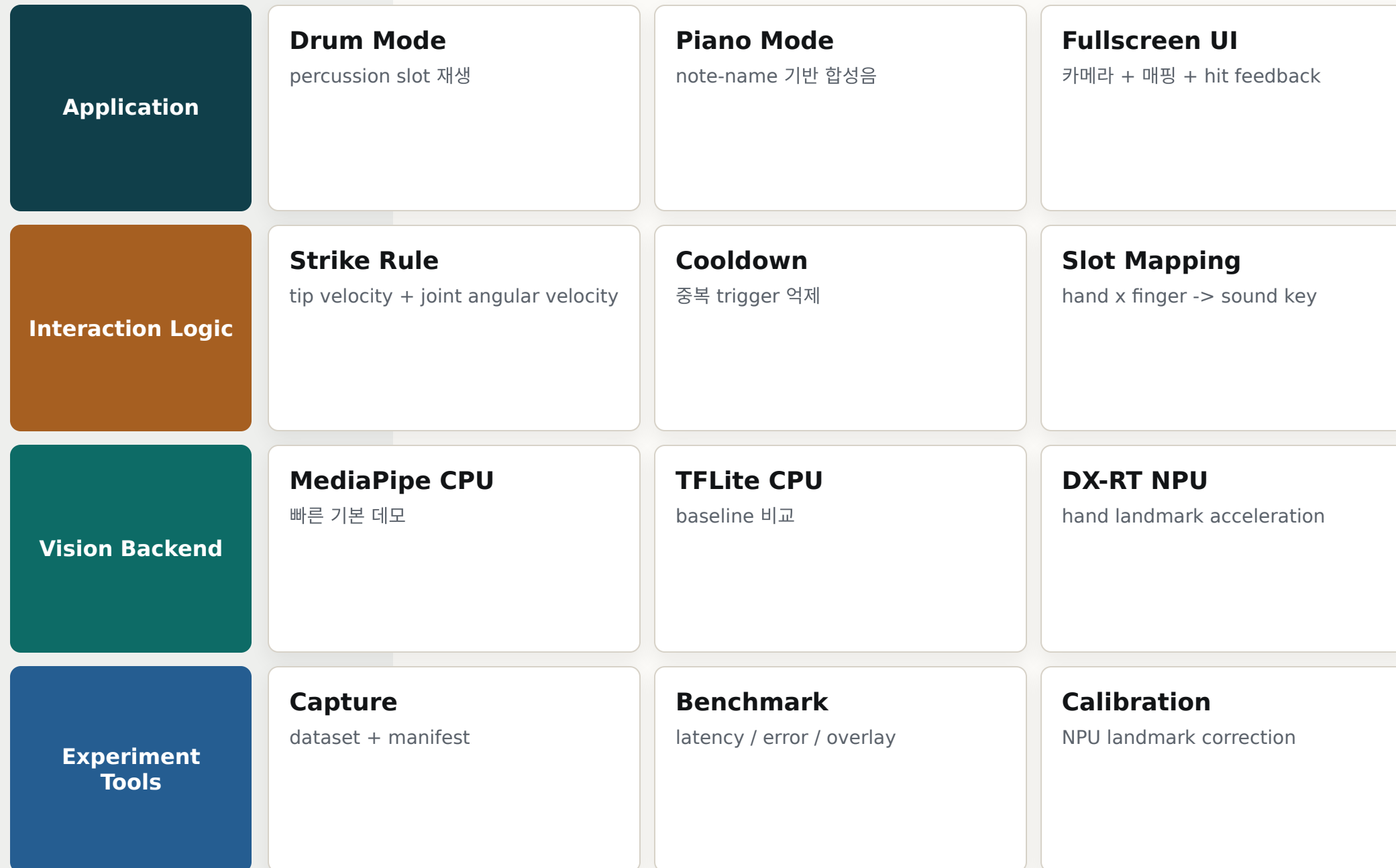
구분	요구사항	아키텍처 영향
입력	USB 카메라 기반, 별도 센서 없음	영상 전처리와 손 검출이 전체 pipeline의 시작점
상호작용	고정 패드 영역 없이 손가락 움직임 자체를 연주로 해석	단순 위치 판정이 아니라 landmark 시간 변화 추적 필요
실시간성	연주 느낌을 위해 낮은 지연과 안정적인 frame loop 필요	Palm/landmark inference의 CPU/NPU 분할이 중요
엣지 실행	보드에서 오프라인 실행, 클라우드 의존 없음	DX-RT, TFLite, OpenCV, pygame 중심으로 구성
검증	실험을 반복 가능하게 측정해야 함	라이브 카메라 외에 dataset replay benchmark 필요

중간발표의 핵심은 “새 알고리즘 제안”이 아니라, 제한된 엣지 환경에서 동작하는 전체 시스템을 설계하고 측정 가능한 구조로 만든 점이다.

전체 시스템 컨텍스트



소프트웨어 계층 구조



CPU와 NPU의 역할 분담

CPU	Camera I/O OpenCV frame capture	Palm Detection TFLite float32, 안정 경로	ROI / Decode anchor decode, NMS, affine warp	Gesture / Audio strike logic, pygame mixer
NPU	Hand Landmark CNN 224x224 hand patch	DXNN Runtime DX-RT InferenceEngine	INT8 Inference DeepX NPU execution	Output Tensor 21 landmark coordinates

Palm detection도 NPU로 시도했지만 INT8 양자화 품질 문제가 있어, 중간발표 기준 안정 경로는 **CPU Palm + NPU Hand**이다.

npu-full의 frame 처리 흐름

01

RGB Frame

OpenCV frame을 tracker 입력으로 변환

02

Palm Tensor

192x192 letterbox, [0,1] float32

03

Palm Decode

2016 SSD anchors, score, weighted NMS

04

Hand ROI

회전 사각형 crop, 224x224 patch 생성

05

Landmark

NPU inference 후 원본 좌표로 역변환

CPU-baseline과의 관계

같은 palm/ROI 경로를 사용하고 hand landmark만 CPU TFLite로 바꿔, NPU 효과를 분리해서 비교한다.

왜 ROI가 중요한가?

hand landmark 모델은 손 중심 crop을 전제로 하므로, palm keypoint에서 안정적인 회전 ROI를 만드는 것이 정확도에 직접 영향을 준다.

4개 backend가 의미하는 아키텍처 변형

BACKEND	구성	장점	한계	발표에서의 역할
cpu	MediaPipe graph 전체 CPU	설치 후 가장 쉽게 데모	NPU 실험 분리 불가	기본 시연 경로
cpu-baseline	Palm TFLite + Hand TFLite	`npu-full` 과 동일 구조	느림	NPU 비교 기준선
npu	화면 좌/우 반분할 + Hand NPU	palm 병목 없음	정확한 손 검출 없음	NPU 속도 확인용
npu-full	Palm CPU + Hand NPU	정식 2-hand pipeline	palm CPU 병목	목표 아키텍처

Palm detection NPU화 실패 원인

속도 관점

- Palm `.dxnn` 추론은 약 12 ms로 CPU TFLite보다 빠름
- 속도만 보면 NPU로 옮기는 것이 맞아 보임
- 하지만 detector는 score가 깨지면 box가 있어도 사용할 수 없음

Palm NPU ~12 ms

Palm CPU ~40-95 ms

정확도 관점

- INT8 양자화 후 score head가 손 검출 점수를 잃음
- ONNX와 NPU score correlation: -0.11
- ema/minmax, opt level 변경으로 해결되지 않음

score head: broken

box head: partially usable

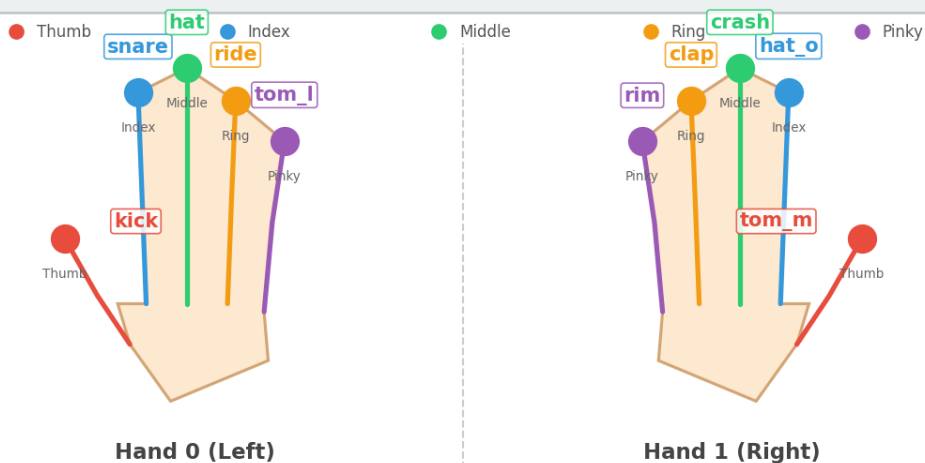
결론: 중간발표 기준 목표 아키텍처는 Palm까지 전부 NPU가 아니라, **Palm은 CPU 안정 경로, Hand landmark는 NPU 가속**이다.

타격 판정은 위치가 아니라 시간 변화 기반

Drum Kit — Default Mapping

10-slot drum: hand × finger → percussion sound

```
$ python3 main.py --camera 0
```



입력 feature

- 각 손의 21개 landmark
- 5개 fingertip 좌표: thumb, index, middle, ring, pinky
- 각 손가락의 MCP/PIP/TIP 계열 관절 각도

타격 조건

- tip y velocity가 아래 방향으로 임계값 이상
- joint angular velocity가 임계값 이상
- cooldown과 최소 변위로 jitter와 중복 hit 억제

아키텍처상 장점

고정 zone 좌표를 튜닝하는 문제가 아니라, 손가락별 동작 event를 악기 slot으로 변환하는 구조가 된다.

악기 매핑 계층

공통 추상화: 10 slot

두 손 x 다섯 손가락을 10개의 logical slot으로 보고, 각 slot이 sound key를 가진다.

손 0	엄지, 검지, 중지, 약지, 소지
손 1	엄지, 검지, 중지, 약지, 소지

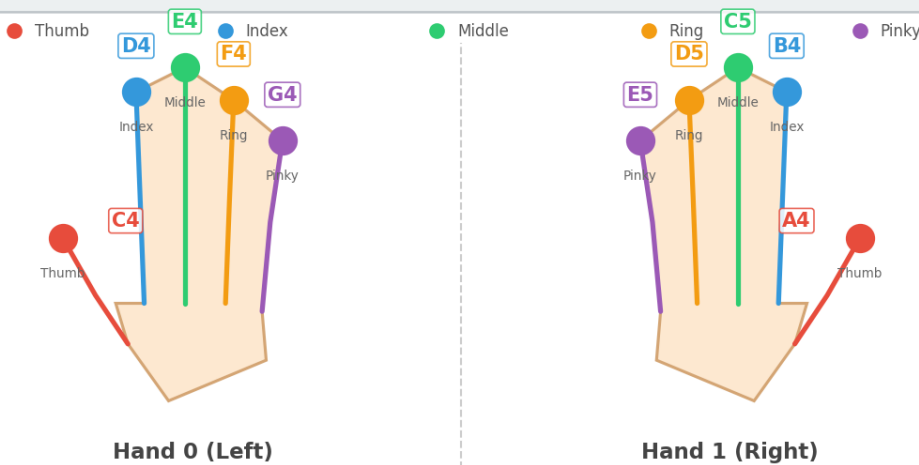
모드별 구현

- Drum: kick, snare, hat, tom, crash 등 합성 percussion
- Piano: C4-E5 등 note-name 기반 합성 tone
- JSON mapping으로 slot 재배치 가능
- UI sidebar에 현재 mapping을 그림으로 표시

Piano — Default C Major (Dynamic)

10-slot piano: C4-E5 pentatonic, distance-adaptive range

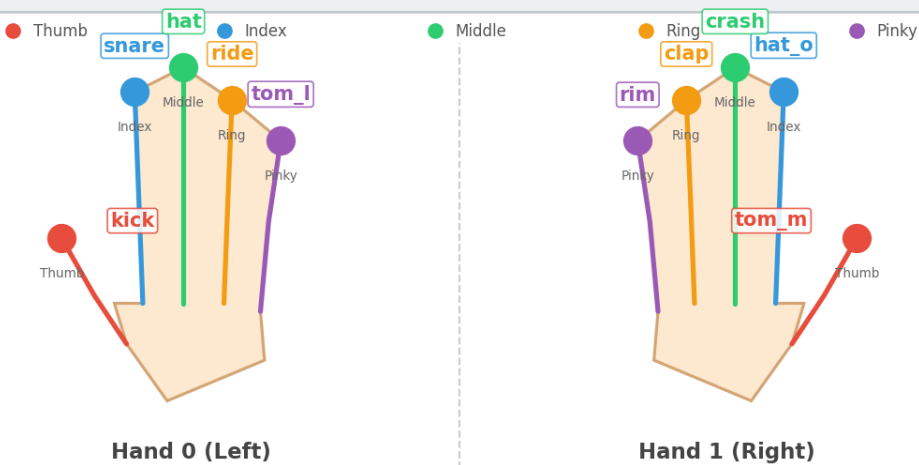
```
$ python3 main.py --piano --camera 0
```



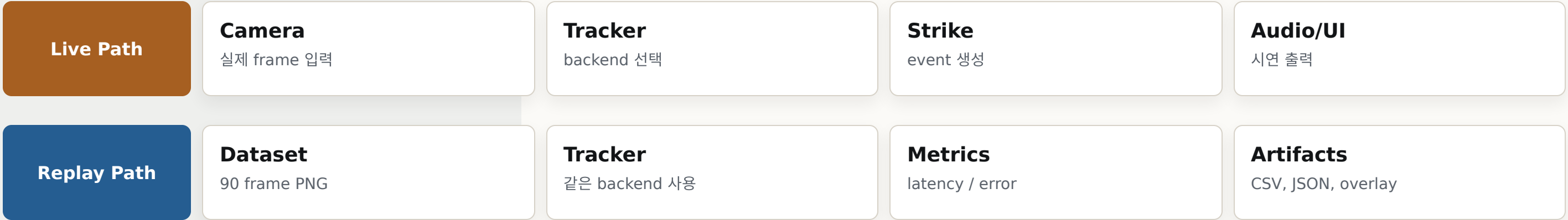
All Available Drum Sounds

16 built-in percussion samples (first 10 mapped by default) | +6 extra: cowbell, shaker, conga, bongo, perc, fx

```
$ python3 main.py --instruments custom.json --camera 0
```



라이브 데모와 오프라인 실험을 분리



이 분리 덕분에 카메라 환경이 매번 달라도, 같은 frame 입력으로 CPU와 NPU backend의 차이를 반복 측정할 수 있다.

DATASET AND METRICS

측정 데이터와 지표

항목	현재 상태	의미
Frame data	90 frames, 640x480 PNG	3개 single-strike session
Reference	cpu-baseline	같은 palm/ROI 구조의 CPU TFLite 기준
Latency	mean, p50, p95, min, max	프레임 처리 시간 비교
Profile	palm_ms, hand_ms, mode	병목 위치 분석
Landmark error	mean / fingertips / max	NPU 좌표 품질 확인



현재 확보한 성능 결과

구성	평균	P95	PROFILE	해석
<code>cpu-baseline</code> , palm every frame	84.40 ms	86.78 ms	palm 39.26 + hand 44.80	전체 CPU 기준
<code>npu-full</code> , palm every frame	50.32 ms	54.52 ms	palm 40.93 + hand 9.13	hand NPU 가속 효과
<code>npu-full</code> , redetect every 5	15.29 ms	50.96 ms	palm 3/20 frames	지연 개선, drift 검증 필요

병목 해석

Palm CPU: 큰 고정 비용

Hand NPU: 줄어든 비용

NPU를 적용해도 Palm이 CPU에 남아 전체 latency를 지배한다.

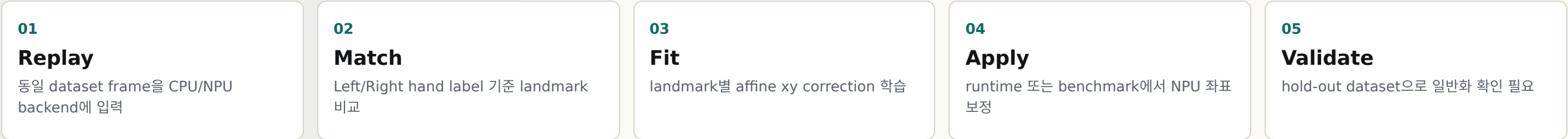
설계 판단

정확도 우선: palm every frame

속도 실험: tracking / async

기본값은 정확도 우선, 최적화는 dataset benchmark로 검증한다.

Landmark 보정 workflow



손	보정 전 MEAN	보정 후 MEAN	보정 전 FINGERTIP	보정 후 FINGERTIP
Right	0.0270	0.0102	0.0336	0.0112
Left	0.0256	0.0092	0.0353	0.0090

현재 값은 같은 dataset에서 fit/eval한 결과이므로, 최종 성능 주장에는 hold-out 검증이 필요하다.

Palm 병목을 줄이는 실험 구조

Palm Redetect Interval

매 프레임 palm detection을 하지 않고, 일정 프레임 동안 이전 landmark에서 ROI를 재추정한다.

- 장점: 평균 latency 감소
- 위험: NPU landmark 편향이 누적되어 ROI drift 발생 가능
- 검증: `0,1,2,3,5,10` sweep으로 latency/error curve 작성

Async Palm

Palm detection은 background thread에서 돌리고, foreground는 이전 ROI로 hand landmark를 계속 추적한다.

- 장점: frame loop의 체감 latency를 줄일 수 있음
- 위험: 빠른 손 움직임에서 stale ROI 사용
- 검증: camera pacing과 실제 live input 모두 비교 필요

두 방법 모두 “기본 아키텍처”가 아니라 최적화 실험이다. 교수/리뷰어에게는 기본값이 정확도 우선임을 먼저 밝히는 것이 안전하다.

모델 자산 생성 흐름

01

MediaPipe TFLite

hand_landmark_lite,
palm_detection_lite

02

ONNX Export

tflite2onnx 또는 flatc 기반 변환

03

DX-COM Config

input layout, quantization, output
mapping

04

Compile Server

x86_64/SNU server에서 `.dxnn` 생성

05

Board Runtime

DX-RT `dx\_engine`으로 NPU 실행

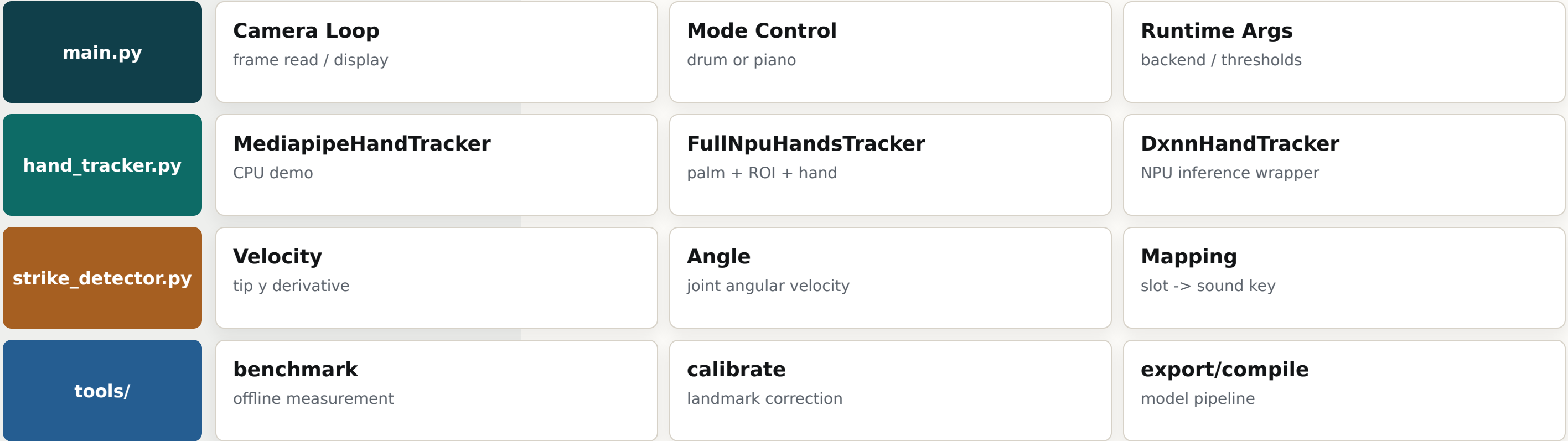
보드 제약

aarch64 보드에서는 DX-COM 컴파일이 직접 어렵고, 컴파일된 `.dxnn`을 복사해 실행하는 구조다.

레이아웃 JSON

DXNN 출력 텐서 순서와 좌표계를 해석하기 위해 `models/dxnn\_layout\*.json`이 필요하다.

코드 모듈 의존 구조



레포 생성 이후 구현 흐름

2026-04-17 Prototype - 손 추적 기반 드럼 - 다중 손/손가락 - 고정 zone 제거 - 피아노 모드	2026-04-17 NPU Path - TFLite -> ONNX - DX-COM wrapper - Hand `.dxnn` - Palm pipeline plan	2026-04-20 Dataset - capture tool 90 frame dataset - ground-truth notes - session documentation	2026-05-08 Experiment Loop - benchmark replay - async/palm sweep - correction - unit tests
44 commits	60 tracked files	8530+ insertions since initial prototype	4 tracking backends

VALIDATION STATUS

현재 검증 상태

18/18

unit tests passed

15/15

palm decode self-test passed

OK

CPU-baseline benchmark smoke

Needs

DX-RT service for NPU smoke

항목	상태	발표에서의 표현
CPU demo path	준비됨	실시간 데모 가능
Dataset replay	준비됨	반복 측정 가능
NPU full runtime	환경 조건 필요	DX-RT service 확인 후 재실행 필요
Palm NPU	실사용 제외	양자화 실패 사례로 설명

리뷰어가 물을 가능성이 높은 지점

1. Palm이 CPU면 NPU 프로젝트인가?

Hand landmark CNN은 NPU로 실행한다. Palm까지 NPU화는 시도했으나 detector score 품질 문제로 안정 경로에서 제외했다.

2. 보정 결과가 overfit 아닌가?

맞다. 현재는 같은 dataset에서 가능성 확인 단계이며, hold-out dataset이 다음 필수 작업이다.

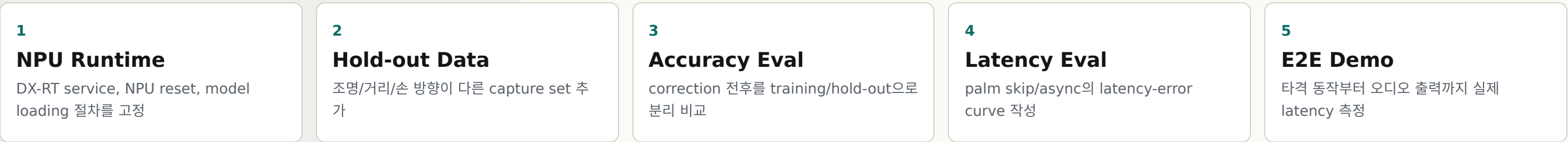
3. offline benchmark가 실제 연주 지연을 대체하나?

대체하지 않는다. vision pipeline 비교용이고, 최종 평가는 camera-to-audio E2E 측정이 필요하다.

4. 최종 데모 전략은?

CPU path로 안정 데모를 확보하고, NPU path는 runtime 복구 후 수치 비교와 함께 보여주는 전략이 안전하다.

최종발표까지의 작업 계획



다음 단계의 목표는 기능 추가보다 **재현성 있는 NPU 데모와 정량 결과**를 확보하는 것이다.

중간발표 결론

현재 프로젝트는 손동작 기반 가상 악기 데모와 CPU/NPU 비교 실험 구조를 모두 갖춘 상태다. 아키텍처상 핵심은 Palm/ROI/Hand를 분리하고, 안정성이 검증된 부분부터 NPU에 배치한 점이다.

구현 완료

air drum / piano, 4개 backend, dataset replay, 단위 테스트

검증 중

NPU runtime 재현성, palm skip/async, landmark correction 일반화

다음 목표

hold-out dataset과 실제 E2E latency 기반 최종 평가